Test Strategy Analysis

Invent.ai - Software Test Engineer Case Study

Candidate	Husnuye
Date	2026-01-28
Scope	API automation (Restful Booker) + UI automation (Sauce Demo) + Scenario mining for discount calculation bug

This document summarizes the overall test strategy, risk analysis, and AI-assisted scenario mining for a reported production issue: **"Total discount is calculated incorrectly when a coupon code is used."**

Repository artifacts (separate from this PDF):

- ApiTests/ (Playwright API tests, schema validation, dynamic data generation)
- WebTests/ (Playwright UI + visual snapshot tests using Page Object Model)
- prompts/prompts.md (prompts used)
- reports/Part3-Scenario-Mining.md and demo videos

1. Strategy Overview

1.1 What we are testing

The reported issue suggests a **pricing/discount calculation defect** triggered when a coupon is applied. This typically involves multiple interacting components: cart pricing rules, promotion stacking, eligibility filtering, rounding, tax/shipping calculation order, and UI/backend consistency.

1.2 Primary risks

- Revenue loss (over-discount) or customer churn (under-discount).
- Fraud/exploit scenarios (e.g., fixed coupon larger than subtotal).
- Inconsistent totals between UI, payment authorization, and invoice/receipt.
- Rounding discrepancies creating repeated customer complaints (0.01 differences).
- Complex rule interactions (caps, thresholds, eligibility subsets).

1.3 Test approach

- **API level**: validate business logic and contracts (schema + type), ensure create flows use dynamic data to avoid false positives.
- **UI level**: validate critical user journeys (checkout) and visual differences (snapshot testing).
- **Scenario mining**: use an LLM to enumerate edge cases, then apply QA judgment to pick the highest-risk tests and refine one into final Gherkin.

2. Scenario Mining for Coupon Discount Bug

2.1 20 edge case scenarios (LLM assisted)

1.	% coupon + existing item-level promotion: order of operations (double-discount).
2.	Fixed-amount coupon exceeds subtotal (negative total / below zero).
3.	Coupon applies only to specific categories with mixed cart.
4.	Coupon applies only to specific SKUs/brands with mixed cart.
5.	Minimum subtotal threshold: applying coupon drops subtotal below threshold (re-check logic).
6.	Maximum discount cap is configured but not enforced correctly.
7.	Free-shipping coupon combined with another coupon (total discount definition ambiguity).
8.	Single-use coupon reused (session/user state mismatch).
9.	First-order-only coupon (new vs returning customer detection).
10.	Tax-inclusive vs tax-exclusive discount base mismatch.
11.	Shipping included vs excluded from discount base mismatch.
12.	Quantity change / item removal after coupon apply (recalculation drift).
13.	Bundles/kits: discount allocation across bundle items incorrect.
14.	Replace coupon A with B: stale discount remains or stacking occurs.
15.	Rounding differences: per-line vs total rounding (0.01 deltas).
16.	Multi-currency conversion errors with fixed value coupons.
17.	Quantity limits (only N items discounted) misapplied.
18.	Sale price already applied + coupon (stacking rules wrong).
19.	Gift card + coupon ordering affects total discount reported.
20.	Split shipment / multi-seller cart: coupon should apply to subset only.

2.2 Top 5 critical scenarios (selected)

Scenario	Why it matters
% coupon + existing item promotion	Highest frequency in real stores; wrong stacking/order causes direct revenue and trust impact.
Fixed coupon > subtotal	Exploit/fraud risk and potential payment/invoice breakage.
Eligibility subset in mixed cart	Most common calculation defects come from item-level allocation and filtering.
Min threshold + max cap interaction	Rule engine complexity; mis-implementation leads to large over/under-discount.

Rounding/precision	Small deltas create high-volume customer complaints and UI/backend mismatches.

2.3 AI gaps and redundancies

Gaps that LLMs commonly miss:

- Discount allocation strategy (pro-rate across items) and how rounding compounds.
- Explicit order of operations: item promos → coupon → tax → shipping (or system-specific).
- State transitions: remove item/change qty/switch coupon should re-evaluate consistently.
- UI vs backend invoice/payment consistency checks.
- Idempotency and concurrency (duplicate apply requests).

Redundant areas: multiple invalid-coupon variants (expired/invalid/usage-limit) are mostly validation; for this bug we prioritize calculation and rule interaction tests.

3. Final Gherkin for the Most Complex Scenario

Selected scenario: mixed cart with item promotion + percent coupon + max cap + minimum threshold + rounding.

```
Feature: Coupon discount calculation

Background:
Given the pricing service is available
And the customer is on the checkout page

Scenario: Correct total discount for mixed cart with item promotion, coupon cap and
minimum threshold
Given the cart contains:
| sku | name | qty | unit_price | item_promo_type | item_promo_value |
| A-001 | Running Shoes | 1 | 120.00 | PERCENT | 10 |
| B-010 | Socks (3-pack) | 2 | 25.00 | NONE | 0 |
And shipping fee is 10.00
And tax is calculated on discounted item totals
And a coupon code "SAVE20" is configured as:
| type | value | applies_to | min_subtotal | max_discount |
| PERCENT_ON_SUBTOTAL | 20 | ELIGIBLE_ITEMS_ONLY | 100.00 | 30.00 |
When the customer applies the coupon code "SAVE20"
Then item-level promotions should be applied before the coupon discount
And the coupon discount should apply only to eligible items after item-level promotions
And the coupon discount should not exceed the maximum discount cap of 30.00
And the subtotal after all discounts should be greater than or equal to 0.00
And the total discount displayed should equal the sum of item-level promotions and
coupon discount
And all monetary values should be rounded to 2 decimal places using standard rounding
And the order total displayed should match the backend invoice total
```

Appendix: Automation implementation details are available in the repository (ApiTests/ and WebTests/).